

```
// === ДОБАВЛЯЕМ ОБЯЗАТЕЛЬНУЮ ВЕРШИННУЮ ТОЧКУ ВХОДА ДЛЯ DIRECTX 11 ===
```

```
struct VS_OUTPUT
```

```
{
```

```
    float4 Pos : SV_POSITION;
```

```
    float2 Tex : TEXCOORD0;
```

```
};
```

```
VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)
```

```
{
```

```
    VS_OUTPUT output = (VS_OUTPUT)0;
```

```
    output.Pos = Pos;
```

```
    output.Tex = Tex;
```

```
    return output;
```

```
}
```

```
// =====
```

```
// ИСТИННЫЙ ОДНОПРОХОДНЫЙ ДЕФЕКТОСКОП АНАТОЛИЯ ФЕДОРОВИЧА (ОБРАЗЕЦ 2011 ГОДА)
```

```
// =====
```

```
// Входные ресурсы DirectX 11
```

```
Texture2D srcTexture : register(t0); // t0: Сюда C++ подает оригинал (строка 970)
```

```
Texture2D macroTexture : register(t1); // t1: Сюда подается маленькая макротекстура 46x46
```

```
Texture2D filterTexture : register(t2); // t2: СЮДА C++ ПОДАЕТ БОЛЬШОЙ КАДР ИЗ СЛОТА 2 (строки 973 и 976)!
```

```
SamplerState samplerState0 : register(s0);
```

```
cbuffer cbData : register(b0)
```

```
{
```

```
    float width;    // p0[0]
```

```
    float height;   // p0[1]
```

```
    float d_width;  // p0[2]
```

```
    float d_height; // p0[3]
```

```
    float splitX;   // Положение шторки
```

```
    float padding;
```

```

float macroWidth; // Принимаем 46.0f от процессора
float macroHeight; // Принимаем 46.0f от процессора
};

static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

// Вспомогательная функция чтения и перевода пикселя в монохром (tex2D)
float GetGray(float2 uv)
{
    //float2 center_uv = uv + float2(padding, padding);
    //return dot(macroTexture.Sample(samplerState0, uv).rgb, WEIGHT);
    //return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
    return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
}

// 1. ШЕЙДЕР ДЛЯ ЛЕВОГО ВЕРХНЕГО УЧАСТКА
float4 func_LT(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===

    float2 tex_in_3 = tex_in;

    // Раздвигаем соседей строго на физический размер макроблока 4x4
    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

    //float2 k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);

```

```

float k_TB = GetGray(tex_TB);
float k_LRTB = GetGray(tex_LRTB);

int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (9 + i_y) +
              (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

// Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
float k1 = GetGray(tex_in);
float delta = k1 - k_SS;

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 2. ШЕЙДЕР ДЛЯ ПРАВОГО ВЕРХНОГО УЧАСТКА
float4 func_RT(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===
    float2 tex_in_3 = tex_in;

```

```

// Раздвигаем соседей строго на физический размер макроблока 4x4
float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

//float2 k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
float k_S = GetGray(tex_in_3);
float k_LR = GetGray(tex_LR);
float k_TB = GetGray(tex_TB);
float k_LRTB = GetGray(tex_LRTB);

int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (9 + i_y * 2) +
              (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (7 - i_y * 2)) / 256.0f;

// Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
float k1 = GetGray(tex_in);
float delta = k1 - k_SS;

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

```

```

// 3. ШЕЙДЕР ДЛЯ ПРАВОГО НИЖНЕГО УЧАСТКА
float4 func_RB(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===

    float2 tex_in_3 = tex_in;

    // Раздвигаем соседей строго на физический размер макроблока 4x4
    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

    //float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);
    float k_TB = GetGray(tex_TB);
    float k_LRTB = GetGray(tex_LRTB);

    int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
    int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

    float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (15 - i_y * 2) +
        (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (1 + i_y * 2)) / 256.0f;

    // Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
    float2 k1 = GetGray(tex_in);
    float delta = k1 - k_SS;

    // Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)

```

```

float g_Gamma = 1.0f;

float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 4. ШЕЙДЕР ДЛЯ LEVContextОГО НИЖНЕГО УЧАСТКА
float4 func_LB(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===

    float2 tex_in_3 = tex_in;

    // Раздвигаем соседей строго на физический размер макроблока 4x4
    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

    //float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);
    float k_TB = GetGray(tex_TB);
    float k_LRTB = GetGray(tex_LRTB);

    int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
    int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

```

```

float k_SS = ((k_S * (9 + i_x * 2) + k_LR * (7 - i_x * 2)) * (15 - i_y * 2) +
              (k_TB * (9 + i_x * 2) + k_LRTB * (7 - i_x * 2)) * (1 + i_y * 2)) / 256.0f;

// Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
float2 k1 = GetGray(tex_in);
float delta = k1 - k_SS;

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// ГЛАВНЫЙ АППАРАТНЫЙ ШЕЙДЕР ФИЛЬТРАЦИИ (БЕЗ ФАНТОМОВ И СЕРОГО НАЛЕТА)
// ГЛАВНЫЙ АППАРАТНЫЙ ШЕЙДЕР ФИЛЬТРАЦИИ (БЕЗ СЕРОГО НАЛЕТА)
float4 PS_Stage3(VS_OUTPUT input) : SV_Target
{
    // 1. Получаем абсолютную целочисленную позицию текущего пикселя кадра (например, 1024,
    768)
    int2 pixel_pos = int2(input.Pos.xy);

    // 2. Вычисляем координаты левого верхнего угла текущего макроблока 4x4
    int block_x = (pixel_pos.x / 4) * 4;
    int block_y = (pixel_pos.y / 4) * 4;

    // 3. Расчет коэффициентов интерполяции Анатолия Федоровича внутри блока
    int i_x = (pixel_pos.x % 4) * 2;

```

```

int i_y = (pixel_pos.y % 4) * 2;

// 4. Ограничиваем координаты строго полезной шириной (width) и высотой (height) кадра
// Чтобы видеокарта физически не могла залезть в серые мусорные поля дополнения 8x8
int4 bounds = int4(0, 0, (int)width - 1, (int)height - 1);

int2 coord_S = clamp(int2(block_x, block_y), bounds.xy, bounds.zw);
int2 coord_LR = clamp(int2(block_x + 4, block_y), bounds.xy, bounds.zw);
int2 coord_TB = clamp(int2(block_x, block_y + 4), bounds.xy, bounds.zw);
int2 coord_LRTB = clamp(int2(block_x + 4, block_y + 4), bounds.xy, bounds.zw);

// Извлекаем честную монохромную яркость четырех смежных блоков напрямую из t0 через
Load
float k_S = dot(srcTexture.Load(int3(coord_S, 0)).rgb, WEIGHT);
float k_LR = dot(srcTexture.Load(int3(coord_LR, 0)).rgb, WEIGHT);
float k_TB = dot(srcTexture.Load(int3(coord_TB, 0)).rgb, WEIGHT);
float k_LRTB = dot(srcTexture.Load(int3(coord_LRTB, 0)).rgb, WEIGHT);

// 5. Двумерная интерполяция яркости
float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (9 + i_y) +
              (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

// 6. Считываем яркость текущего пикселя кадра и считаем локальную разность
int2 coord_curr = clamp(pixel_pos, bounds.xy, bounds.zw);
float k1 = dot(srcTexture.Load(int3(coord_curr, 0)).rgb, WEIGHT);
float delta = k1 - k_SS;

// Финальный контрастный результат дефектоскопа
float out_gray = saturate(k_SS + delta);
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// ШЕЙДЕР СКВОЗНОГО КОПИРОВАНИЯ ЭКРАНА С УЧЕТОМ ШТОРКИ ДО/ПОСЛЕ

```

```

float4 PS_Final(VS_OUTPUT input) : SV_Target
{
    // =====
    // ШАГ 6: ОТБРАСЫВАЕМ ДОПОЛНЕННЫЕ СТОЛБЦЫ И СТРОКИ (УСЕЧЕНИЕ МУСОРА)
    // =====
    // width/height — реальные размеры картинки (например, 4125 x 3091)
    // macroWidth/macroHeight — расширенные размеры, кратные 8 (например, 4128 x 3096)
    float scaleX = width / macroWidth;
    float scaleY = height / macroHeight;

    // Пересчитываем сквозные координаты экрана [0..1] строго в полезную зону текстур
    float2 cleanUV = float2(input.Tex.x * scaleX, input.Tex.y * scaleY);

    // Страховочный зажим: гарантирует, что из-за округлений мы не зацепим мусорные пиксели на
    // краях
    cleanUV = clamp(cleanUV, float2(0.0f, 0.0f), float2(scaleX - 0.0001f, scaleY - 0.0001f));

    // =====
    // СБОРКА И СИНХРОНИЗАЦИЯ ШТОРКИ РАЗДЕЛЕНИЯ ЭКРАНА
    // =====

    // 1. ЛЕВАЯ ПОЛОВИНА: Исходный недеформированный оригинал в монохроме
    if (input.Tex.x < splitX)
    {
        float4 srcColor = srcTexture.Sample(samplerState0, cleanUV);
        float gray = dot(srcColor.rgb, WEIGHT);
        return float4(gray, gray, gray, 1.0f);
    }

    // 2. РАЗДЕЛИТЕЛЬНАЯ ЛИНИЯ: Тонкая черная вертикальная шторка
    if (abs(input.Tex.x - splitX) < 0.0015f)
    {

```

```

        return float4(0.0f, 0.0f, 0.0f, 1.0f);
    }

    // =====
    // ТОЧНОЕ ИСПРАВЛЕНИЕ: Читаем строго из filterTexture по единому масштабу!
    // =====
    float4 filteredColor = filterTexture.Sample(samplerState0, cleanUV);
    return float4(filteredColor.rgb, 1.0f);
}

// Простой шейдер копирования кадра для ЭТАПА II
float4 PS_Copy(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// ВРЕМЕННАЯ ЗАГЛУШКА ДЛЯ ИНИЦИАЛИЗАЦИИ ОБЪЕКТА В MAIN.CPP
float4 PS(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// === ОФИЦИАЛЬНЫЕ ЗАГЛУШКИ ДЛЯ ЧИСТОГО СТАРТА ПРОГРАММЫ БЕЗ ОШИБОК ===

float4 PS_Downsample2X(VS_OUTPUT input) : SV_Target
{
    return srcTexture.Sample(samplerState0, input.Tex);
}

// === НАСТОЯЩЕЕ АППАРАТНОЕ СЖАТИЕ БЛОКОВ 4x4 В SHADERS.HLSL ===
float4 PS_Stage2(VS_OUTPUT input) : SV_Target
{

```

```

float4 color_sum = float4(0.0f, 0.0f, 0.0f, 0.0f);

// Запускаем цикл по всему макроблоку 4x4 пикселя
for (int x = 0; x < 4; x++)
{
    for (int y = 0; y < 4; y++)
    {
        // Вычисляем точный сдвиг для каждого из 16 пикселей внутри блока
        // d_width и d_height — это наши шаги константного буфера
        float2 uv_offset = input.Tex + float2(x * d_width, y * d_height);

        // Суммируем цвета всех точек
        color_sum += srcTexture.Sample(samplerState0, uv_offset);
    }
}

// Честно делим сумму на 16 пикселей, получая идеальное среднее значение фона!
return color_sum / 16.0f;
}

```

```

float4 PS_PixelShaderStage2(VS_OUTPUT input) : SV_Target
{
    float4 color_sum = float4(0.0f, 0.0f, 0.0f, 0.0f);
    for (int x = 0; x < 4; x++)
    {
        for (int y = 0; y < 4; y++)
        {
            float2 uv_offset = input.Tex + float2(x * d_width, y * d_height);
            color_sum += srcTexture.Sample(samplerState0, uv_offset);
        }
    }
    return color_sum / 16.0f;
}

```

}